

Scalable Web Application with Load Balancing

Project: Scalable Web Application with Load Balancing

Services: EC2, Elastic Load Balancer (ALB), Auto Scaling, RDS, S3, Route 53 (Concept Only)

Description: Deploy a scalable web application that automatically distributes traffic across multiple EC2 instances using an Application Load Balancer (ALB). The application uses Amazon RDS for database management and Amazon S3 for static file hosting. Auto Scaling automatically maintains healthy EC2 instances and scales resources based on load.

Skills: VPC, Security Groups, EC2 Launch Templates, User Data Scripts, Auto Scaling, Load Balancing, RDS Connectivity, S3 Integration.

Definition: The Scalable Web Application with Load Balancing project is designed to demonstrate a highly available and scalable cloud-native architecture on AWS. The application is hosted on EC2 instances running Apache and PHP, with traffic distributed using an Application Load Balancer (ALB). Auto Scaling automatically launches or replaces EC2 instances based on health status and scaling policies.

Amazon RDS MySQL is used as the backend database service, while Amazon S3 stores static files such as images. The infrastructure is deployed inside a custom VPC using public subnets, security groups, launch templates, and user data automation scripts.

Instead of purchasing a custom domain using Route 53, the project uses the free ALB DNS endpoint to avoid additional domain registration costs.

Scope of the Project: The scope of this project is to design and implement a scalable and highly available web application architecture using AWS cloud services. The system leverages EC2, ALB, Auto Scaling, RDS, and S3 to provide a cloud-native scalable infrastructure.

This project covers:

- **Scalable Web Hosting:** Deploys PHP web applications on EC2 instances managed through Auto Scaling Groups.
- **Load Balancing:** Uses an Application Load Balancer to distribute incoming traffic across multiple EC2 instances.
- **Database Integration:** Uses Amazon RDS MySQL for centralized relational database management.
- **Static File Hosting:** Uses Amazon S3 to store and serve static assets such as images.
- **Automated EC2 Provisioning:** Uses Launch Templates and User Data scripts to automate server configuration.
- **High Availability:** Deploys resources across multiple Availability Zones using multiple subnets.

- **Fault Tolerance:** Automatically replaces unhealthy EC2 instances using Auto Scaling health checks.

Methodology:

1.Auto Scaling + Application Load Balancer – Best Method

- Uses all required AWS services: EC2, ALB, Auto Scaling, RDS, and S3.
- Provides automatic traffic distribution and automatic instance recovery.
- Fully automated server deployment using Launch Templates and User Data scripts.
- High availability architecture using multiple subnets and Availability Zones.
- Production-ready and suitable for scalable cloud deployments.
- Cost-effective and Free Tier optimized for beginner projects.
- Eliminates the need for manual server provisioning and traffic management.

2.EC2 Standalone Deployment – Alternative Method

- Uses a single EC2 instance for hosting the application.
- Simpler setup but lacks scalability and high availability.
- No automatic traffic distribution or automatic recovery.
- Requires manual server maintenance and configuration.
- Suitable only for small-scale or testing environments.
- Higher risk of downtime if the EC2 instance fails.

AWS Services Used in the Project:

1. Amazon EC2 (Elastic Compute Cloud)

- **Definition:** Amazon EC2 is a web service that provides resizable virtual servers in the cloud for hosting applications and workloads.
- **Purpose:** Hosts the PHP web application using Apache Web Server.
- **Role:** Runs the web application instances managed by Auto Scaling Groups.

2.Elastic Load Balancer (Application Load Balancer)

- **Definition:** Elastic Load Balancing automatically distributes incoming application traffic across multiple EC2 instances.
- **Purpose:** Distributes incoming user traffic across healthy EC2 instances.
- **Role:** Improves scalability, fault tolerance, and high availability.

3.Amazon EC2 Auto Scaling

- **Definition:** Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances based on demand and health conditions.
- **Purpose:** Maintains application availability and automatically replaces unhealthy instances.

- **Role:** Launches new EC2 instances using Launch Templates and scaling policies.

4. Amazon RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed relational database service that simplifies database setup, operation, and scaling.
- **Purpose:** Stores application data using MySQL database engine.
- **Role:** Provides centralized and managed database connectivity for the application.

5. Amazon S3 (Simple Storage Service)

- **Definition:** Amazon S3 is an object storage service designed for scalable and durable storage.
- **Purpose:** Stores static files such as images used in the application.
- **Role:** Provides scalable and durable static file hosting independent of EC2 instances.

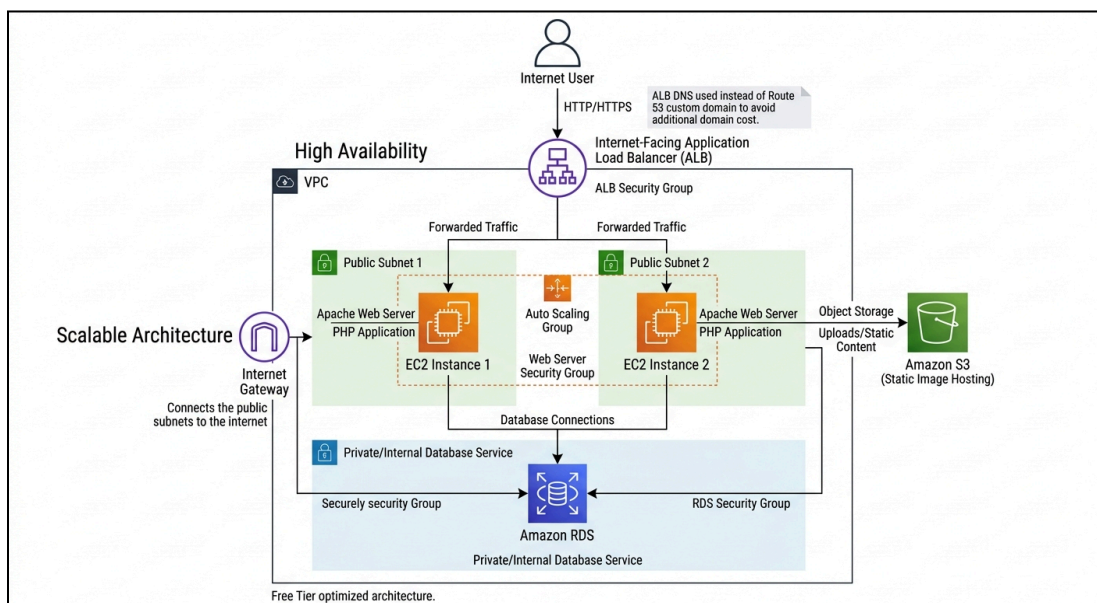
6. Amazon VPC (Virtual Private Cloud)

- **Definition:** Amazon VPC allows users to create isolated virtual networks within AWS.
- **Purpose:** Provides secure networking infrastructure for AWS resources.
- **Role:** Hosts EC2 instances, ALB, and RDS resources inside isolated subnets.

7. Security Groups

- **Definition:** Security Groups act as virtual firewalls controlling inbound and outbound traffic.
- **Purpose:** Protects AWS resources by controlling network access.
- **Role:** Allows controlled communication between ALB, EC2, and RDS.

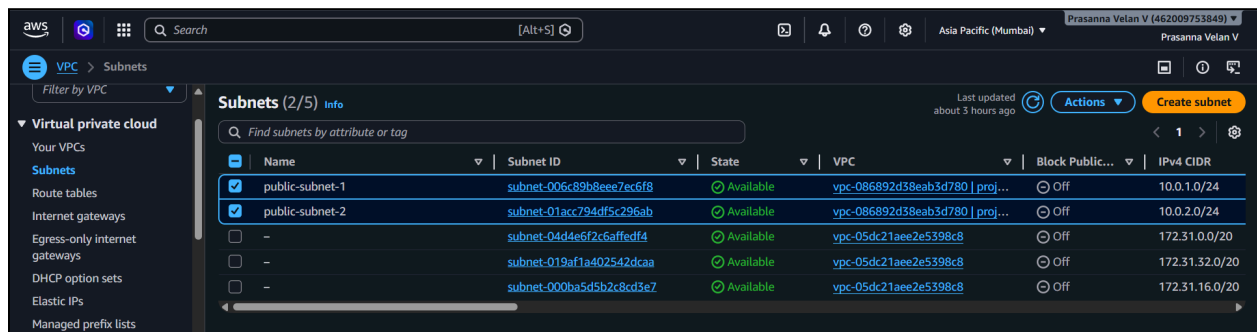
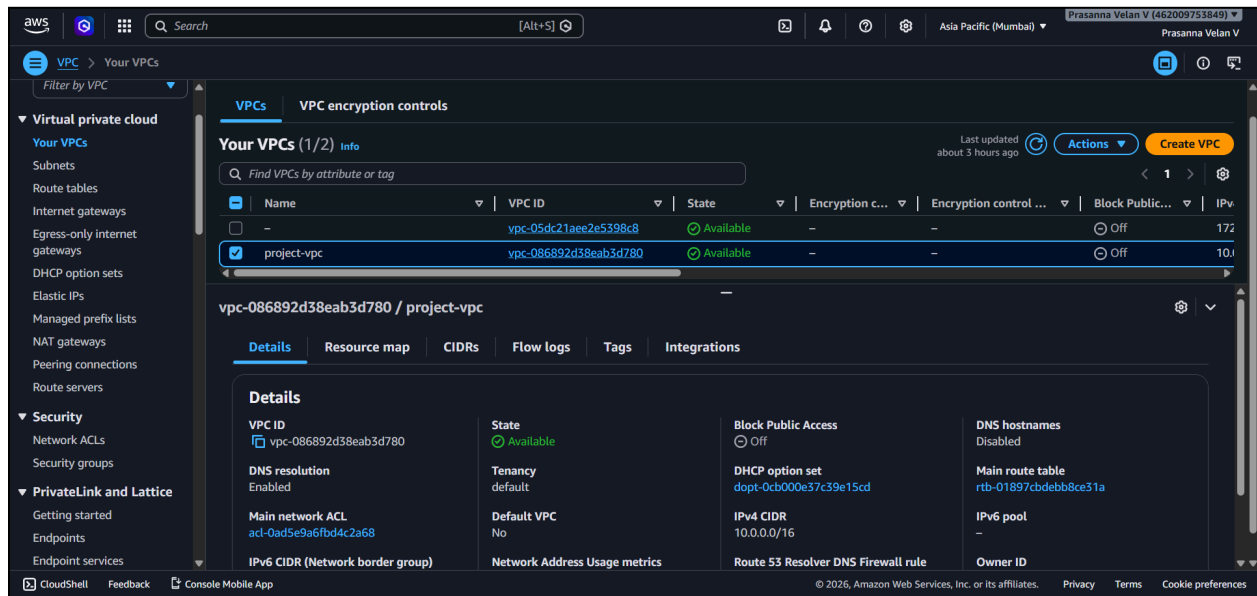
Architecture Diagram:



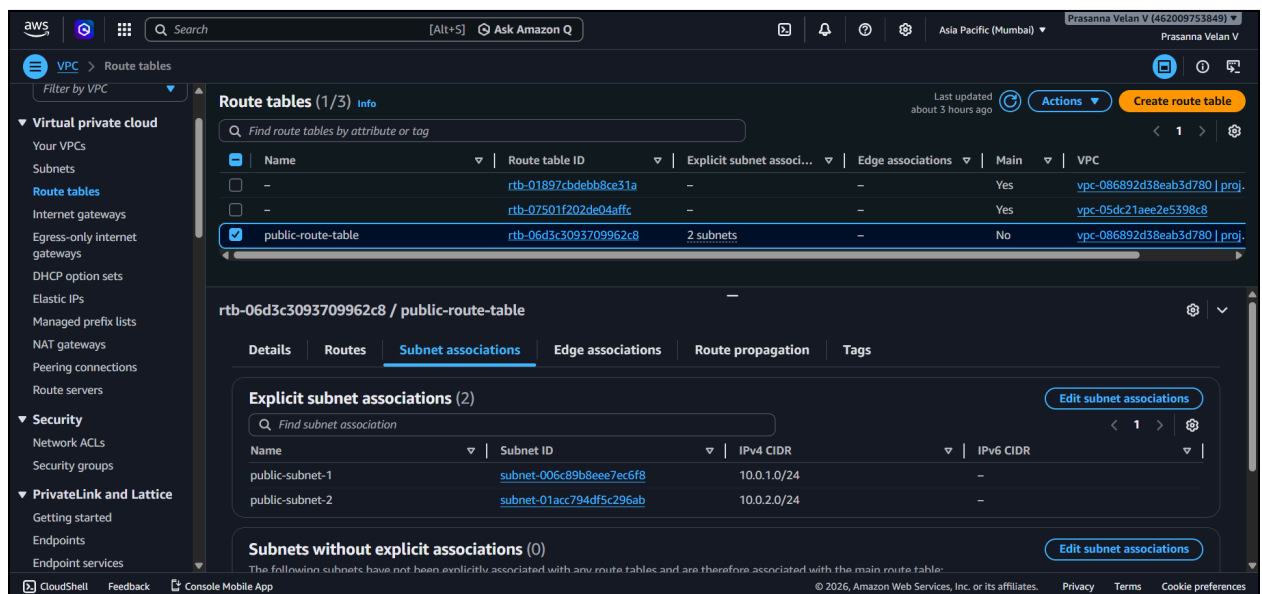
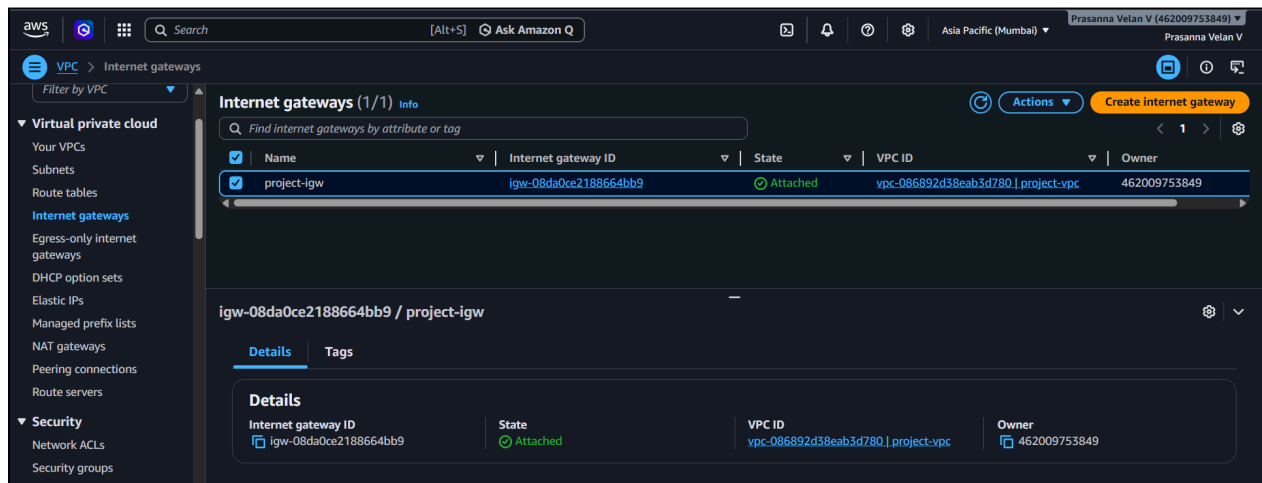
Implementation Steps to Do the Project:

Step 1: Create Custom VPC

- Go to AWS Console → VPC → Create VPC
- Create custom VPC with CIDR block.
- Create: Public Subnet 1 and Public Subnet 2
- Attach Internet Gateway.
- Configure Route Tables for internet access.



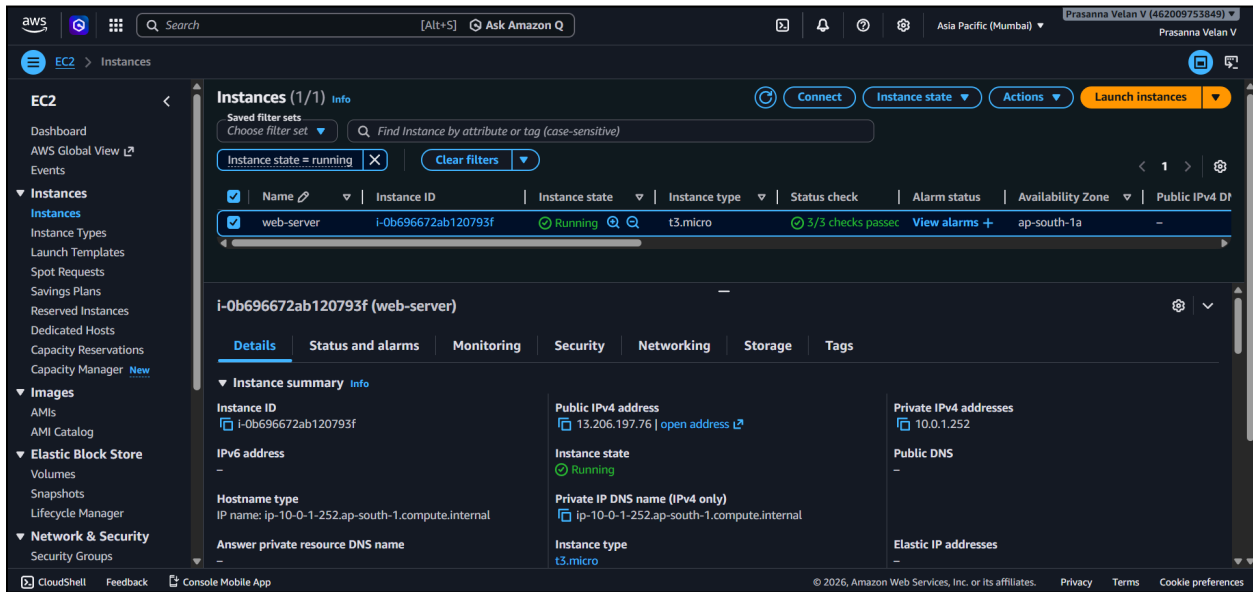
Scalable Web Application with Load Balancing



Step 2: Launch EC2 Web Server

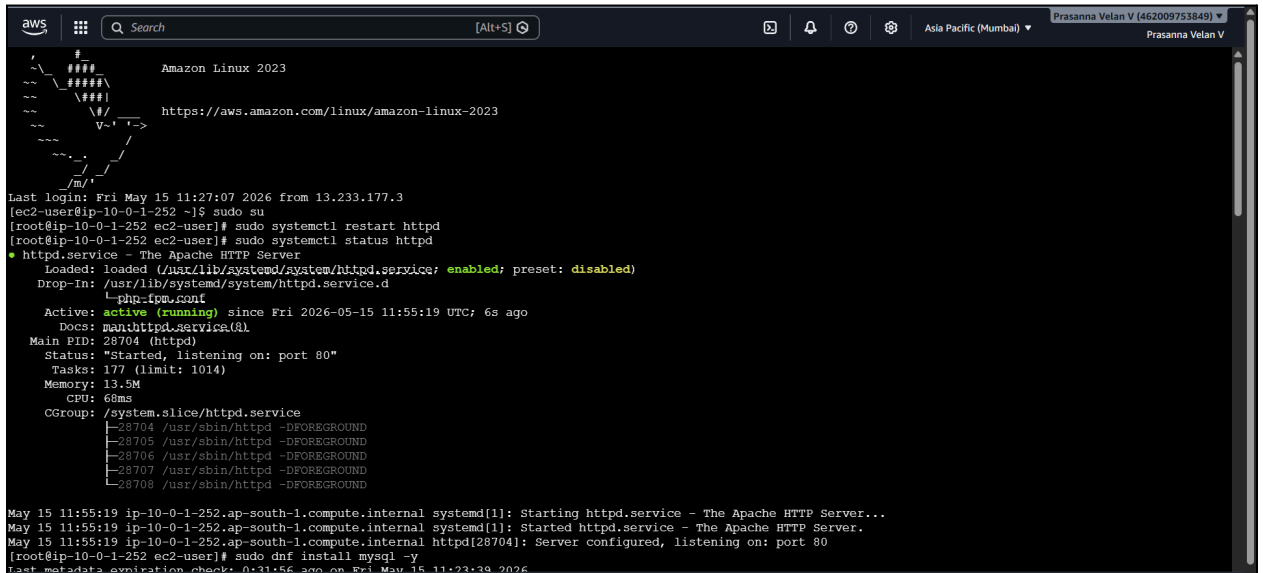
- Go to AWS Console → EC2 → Launch Instances.
- Choose Amazon Linux AMI. Configure: Custom VPC Public Subnet
- Auto-assign Public IP and Create Security Group: HTTP access , SSH access
- Launch EC2 instance using key pair.

Scalable Web Application with Load Balancing



Step 3: Install Apache and PHP

- Connect to EC2 using SSH.
- Install: Apache Web Server , PHP
- MySQL client: Create PHP web application page. Start and enable Apache service.

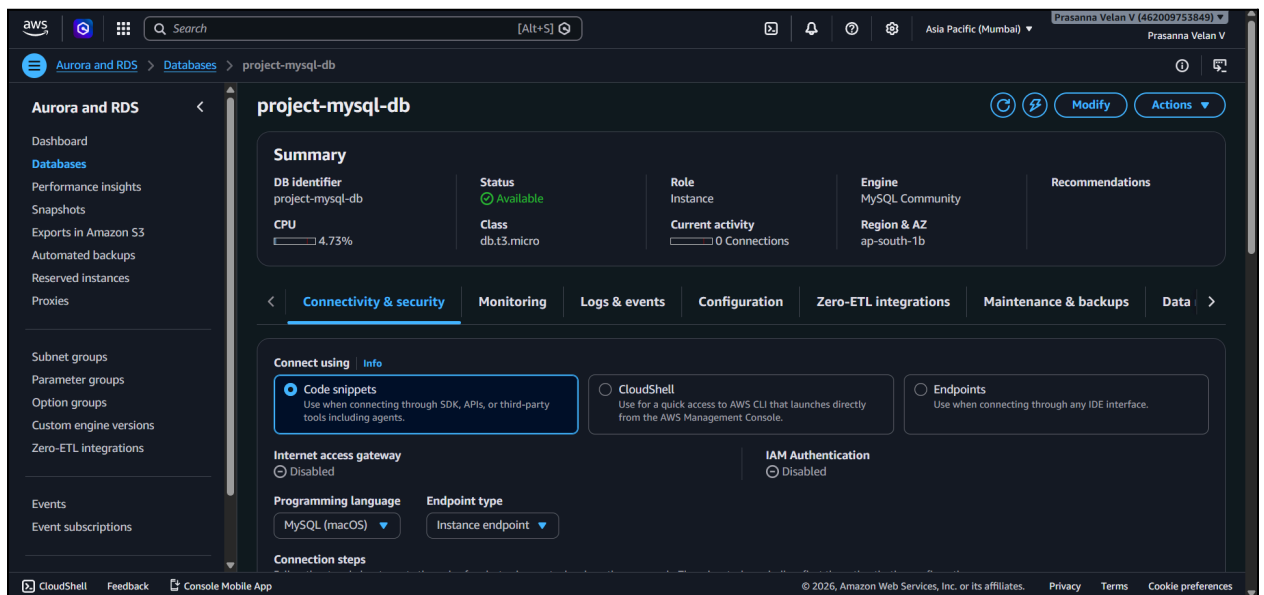
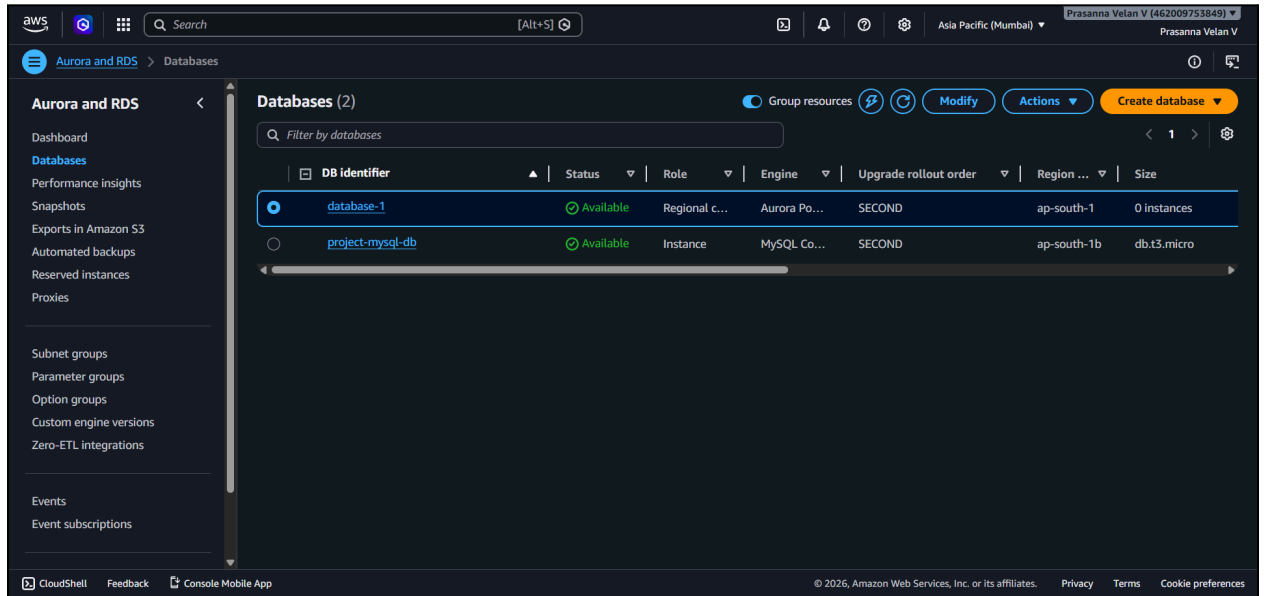


Step 4: Create Amazon RDS MySQL Database

- Go to RDS → Create Database.
- Select MySQL engine.
- Use Free Tier configuration.

Scalable Web Application with Load Balancing

- Disable Multi-AZ deployment.
- Create database security group.
- Configure database connectivity with EC2.



Scalable Web Application with Load Balancing

```
[root@ip-10-0-1-252 ec2-user]# mysql -h project-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 42
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

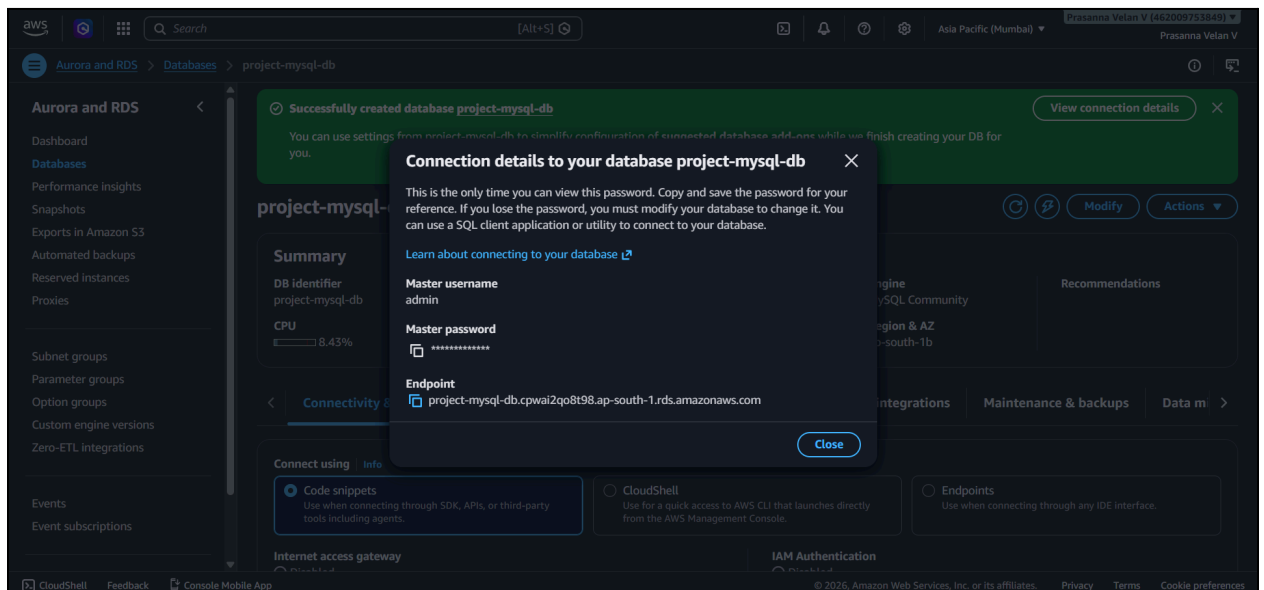
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> INSERT INTO visitors (message)
    -> VALUES ('Auto Scaling project running');
ERROR 1046 (3D000): No database selected
MySQL [(none)]> USE projectdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [projectdb]> INSERT INTO visitors (message)
    -> VALUES ('Auto Scaling project running');
Query OK, 1 row affected (0.011 sec)

MySQL [projectdb]> INSERT INTO visitors (message)
    -> VALUES ('AWS Load Balancer demo');
Query OK, 1 row affected (0.005 sec)

MySQL [projectdb]> cd /home/ec2-user
    -> ^DBye
[root@ip-10-0-1-252 ec2-user]# cd /var/www/html
[root@ip-10-0-1-252 html]# cd /var/www/html
[root@ip-10-0-1-252 html]# sudo nano index.php
[root@ip-10-0-1-252 html]# sudo systemctl restart httpd
[root@ip-10-0-1-252 html]# sudo systemctl status httpd
```



Step 5: Connect PHP Application to RDS

- Update PHP application with RDS endpoint and credentials.
- Create sample MySQL table and insert records.
- Verify database connectivity from application.

Scalable Web Application with Load Balancing

```
[root@ip-10-0-1-252 ec2-user]# mysql --version
mysql Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using EditLine wrapper
[root@ip-10-0-1-252 ec2-user]# mysql -h project-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 32
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> USE projectdb;
Database changed
MySQL [projectdb]> CREATE TABLE visitors (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   message VARCHAR(255),
  ->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.059 sec)

MySQL [projectdb]> INSERT INTO visitors (message)
  -> VALUES ('Database connection successful');
Query OK, 1 row affected (0.028 sec)

MySQL [projectdb]> SELECT * FROM visitors;
+-----+-----+-----+
| id | message                               | created_at |
+-----+-----+-----+
| 1 | Database connection successful | 2026-05-15 12:05:14 |
+-----+-----+-----+
1 row in set (0.002 sec)
```

AWS Scalable Web Application

Welcome to your AWS Project

Server Hostname: ip-10-0-1-252.ap-south-1.compute.internal

Server IP: 10.0.1.252

Date & Time: 2026-05-15 12:18:45

Database Status: Database Connected Successfully

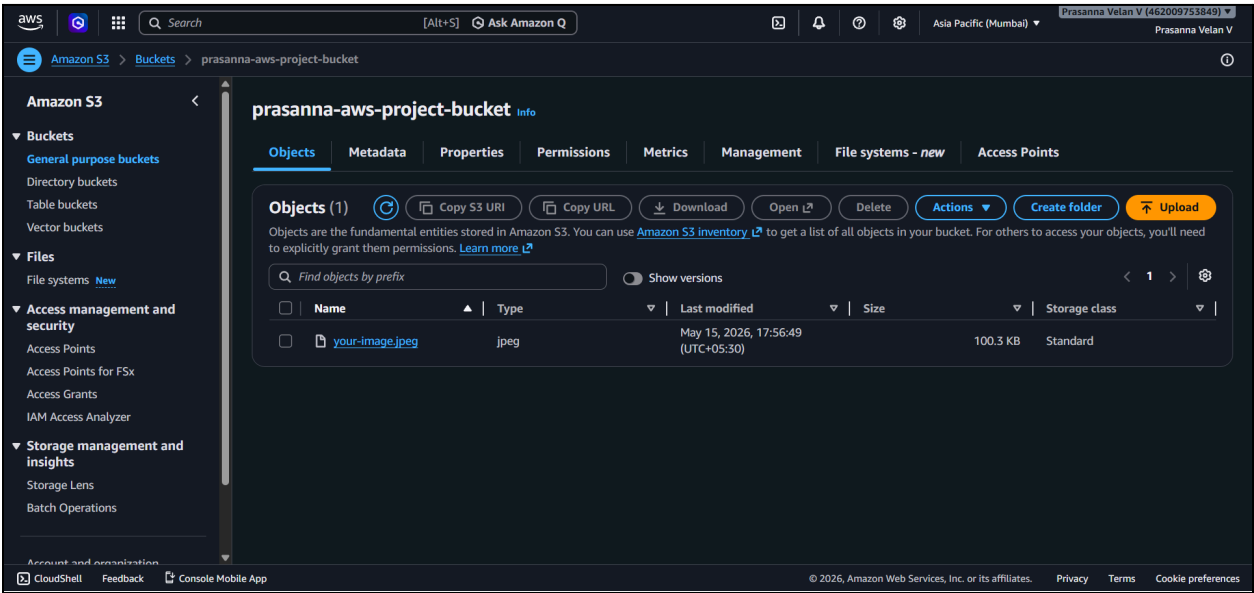
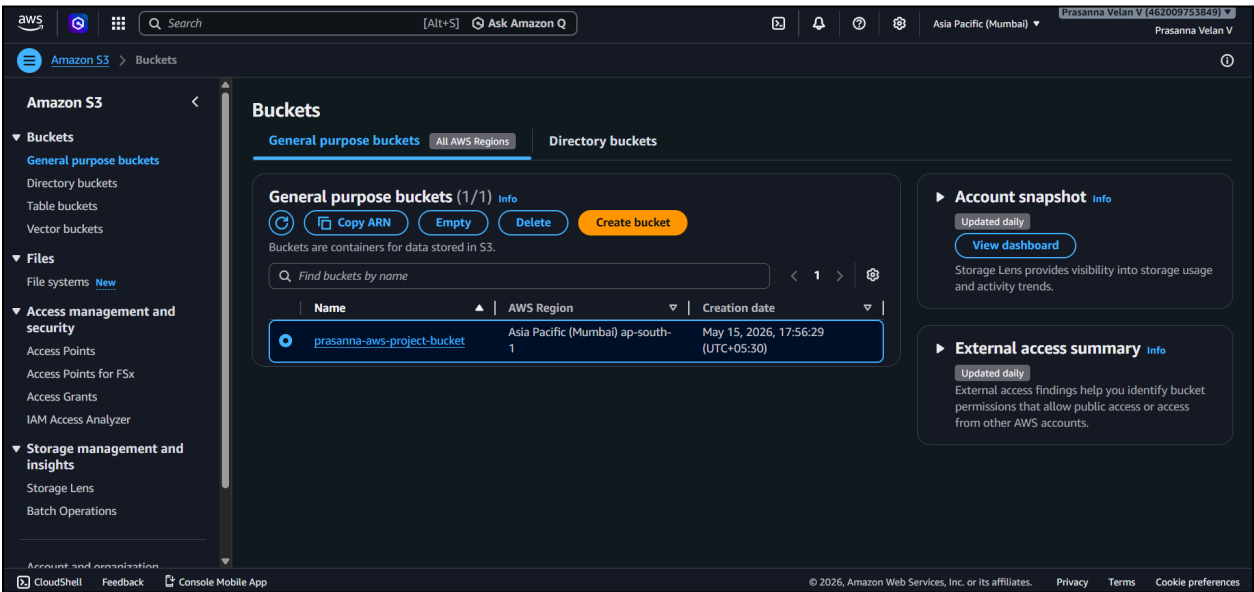
Visitor Table Data

ID	Message	Created At
1	Database connection successful	2026-05-15 12:05:14
2	Auto Scaling project running	2026-05-15 12:18:34
3	AWS Load Balancer demo	2026-05-15 12:18:41

Step 6: Create Amazon S3 Bucket

- Go to S3 → Create Bucket.
- Upload static image files.
- Configure public access for required objects.
- Integrate S3 object URL into PHP application.

Scalable Web Application with Load Balancing



Scalable Web Application with Load Balancing

```
aws | Search | [Alt+S] | index.php | Prasanna Velan V (462009753849) | Prasanna Velan V
GNU nano 8.3
$?php
$hostname = gethostname();
$server_ip = $_SERVER['SERVER_ADDR'];
$date = date("Y-m-d H:i:s");

/* RDS Configuration */
$db_host = "project-mysql-db.cpwal2qo8t98.ap-south-1.rds.amazonaws.com";
$db_user = "admin";
$db_password = "ProjectDB123!";
$db_name = "projectdb";

/* Connect to MySQL */
$conn = new mysqli($db_host, $db_user, $db_password, $db_name);

/* Check connection */
if ($conn->connect_error) {
    $db_status = "Database Connection Failed: " . $conn->connect_error;
} else {
    $db_status = "Database Connected Successfully";

    /* Fetch sample data */
    $sql = "SELECT * FROM visitors";
    $result = $conn->query($sql);
}

?>

<!DOCTYPE html>
<html>
<head>
    <title>AWS Scalable Web App</title>

    <style>
```

```
aws | Search | [Alt+S] | index.php | Prasanna Velan V (462009753849) | Prasanna Velan V
GNU nano 8.3
<h1>AWS Scalable Web Application</h1>

<p class="info">
Welcome to your AWS Project
</p>

<p class="info">
<b>Server Hostname:</b> <?php echo $hostname; ?>
</p>

<p class="info">
<b>Server IP:</b> <?php echo $server_ip; ?>
</p>

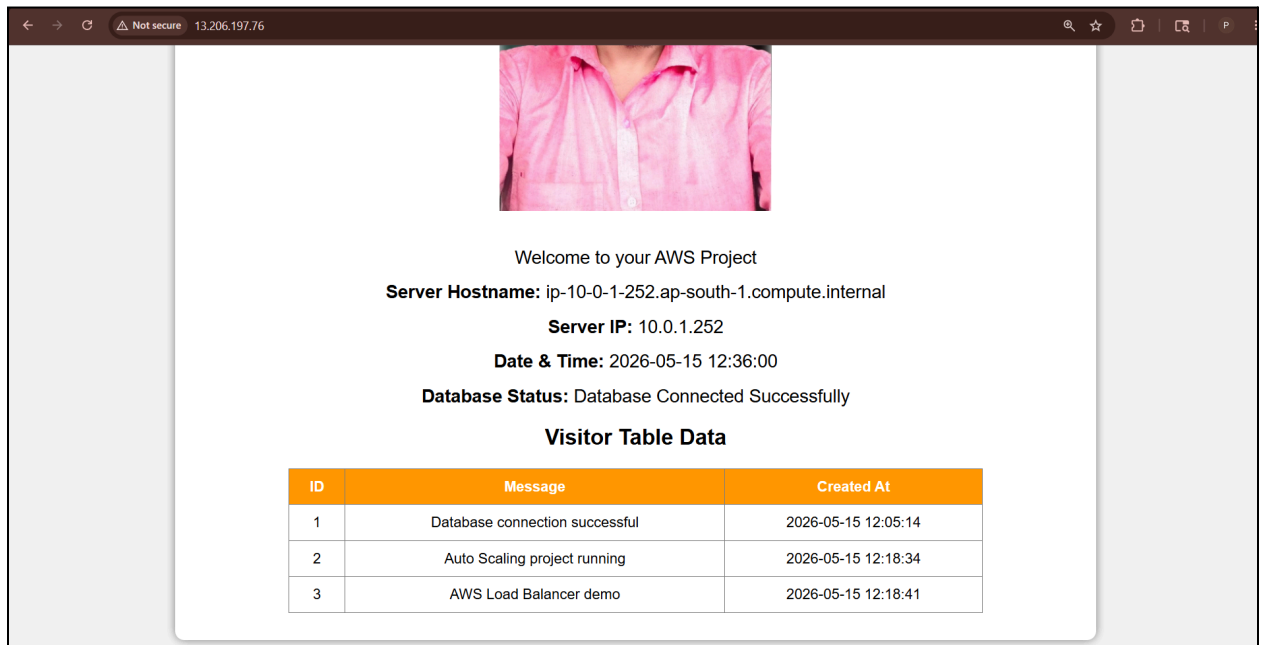
<p class="info">
<b>Date & Time:</b> <?php echo $date; ?>
</p>

<p class="info">
<b>Database Status:</b> <?php echo $db_status; ?>
</p>

<h2>Visitor Table Data</h2>

<table>
<tr>
    <th>ID</th>
    <th>Message</th>
    <th>Created At</th>
</tr>
```

Scalable Web Application with Load Balancing



Welcome to your AWS Project

Server Hostname: ip-10-0-1-252.ap-south-1.compute.internal

Server IP: 10.0.1.252

Date & Time: 2026-05-15 12:36:00

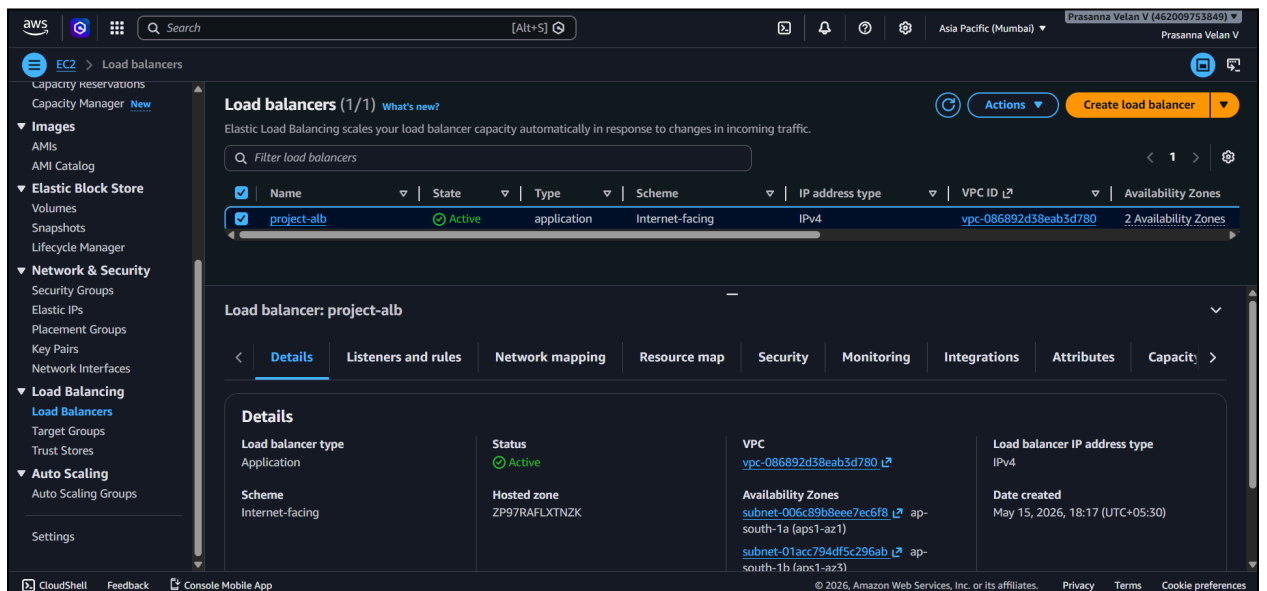
Database Status: Database Connected Successfully

Visitor Table Data

ID	Message	Created At
1	Database connection successful	2026-05-15 12:05:14
2	Auto Scaling project running	2026-05-15 12:18:34
3	AWS Load Balancer demo	2026-05-15 12:18:41

Step 7: Create Application Load Balancer

- Go to EC2 → Load Balancers → Create ALB.
- Configure: Internet-facing ALB ,Public Subnets ,ALB Security Group
- Create Target Group.
- Register EC2 instance.



Load balancers (1/1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
☒	project-alb	Active	application	Internet-facing	IPv4	vpc-086892d38eab3d780	2 Availability Zones

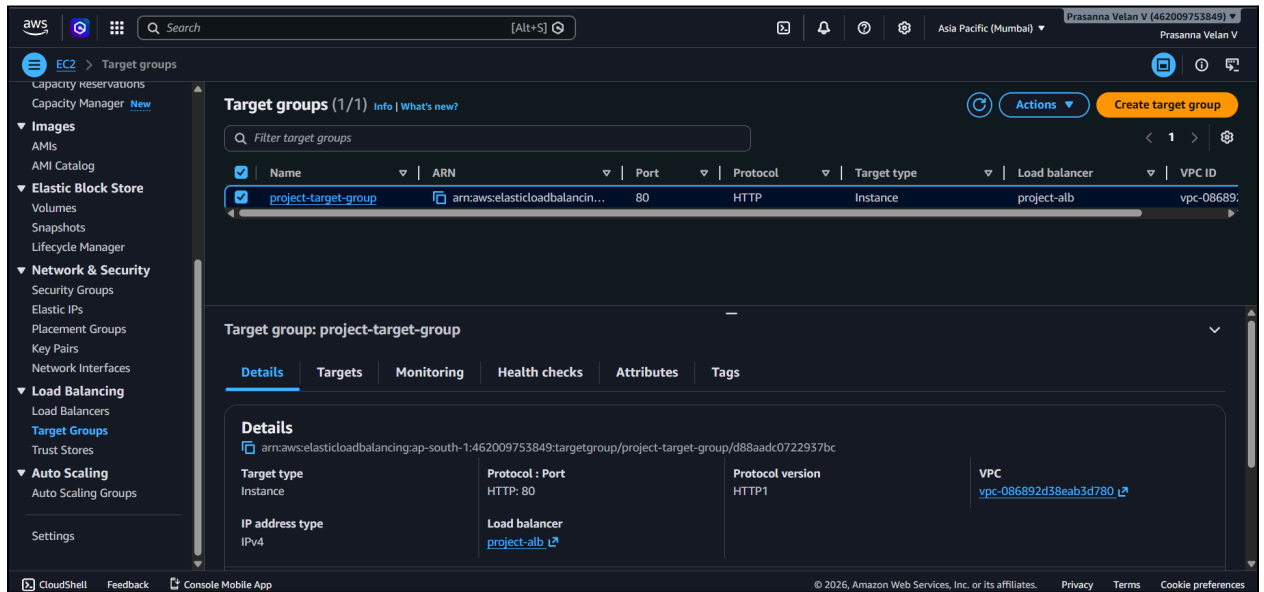
Load balancer: project-alb

[Details](#) [Listeners and rules](#) [Network mapping](#) [Resource map](#) [Security](#) [Monitoring](#) [Integrations](#) [Attributes](#) [Capacity](#)

Details

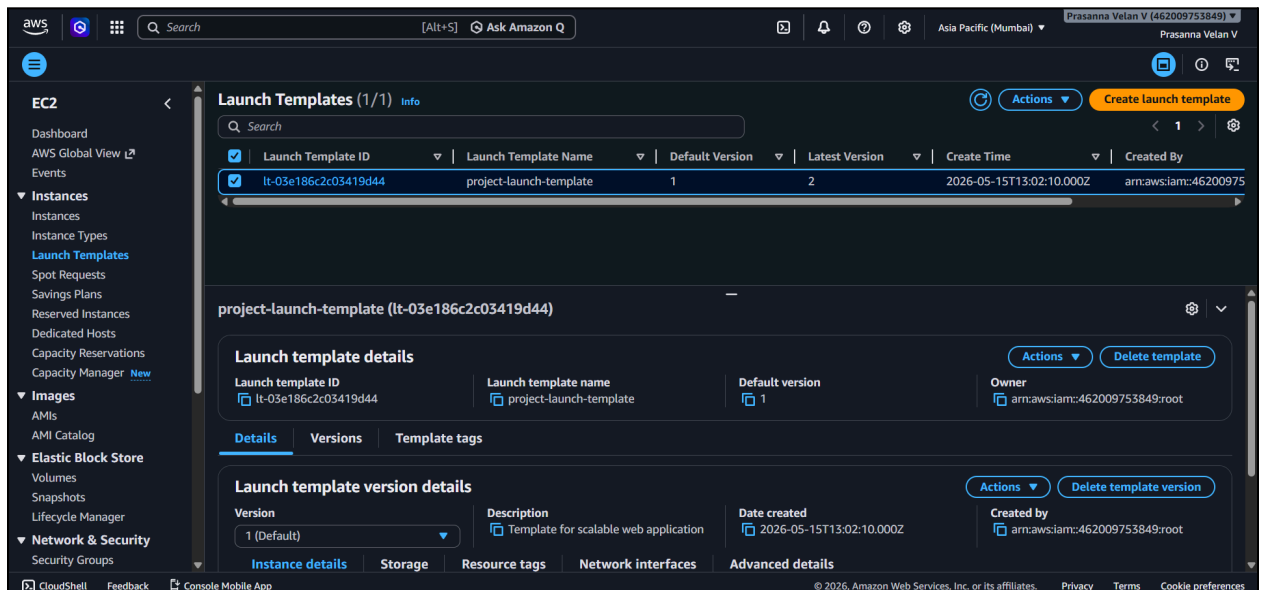
Load balancer type Application	Status Active	VPC vpc-086892d38eab3d780	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone ZP97RAFLXTNZK	Availability Zones subnet-006c89b8ee7ec6f8 ap-south-1a (aps1-az1) subnet-01acc794df5c296ab ap-south-1b (aps1-az3)	Date created May 15, 2026, 18:17 (UTC+05:30)

Scalable Web Application with Load Balancing



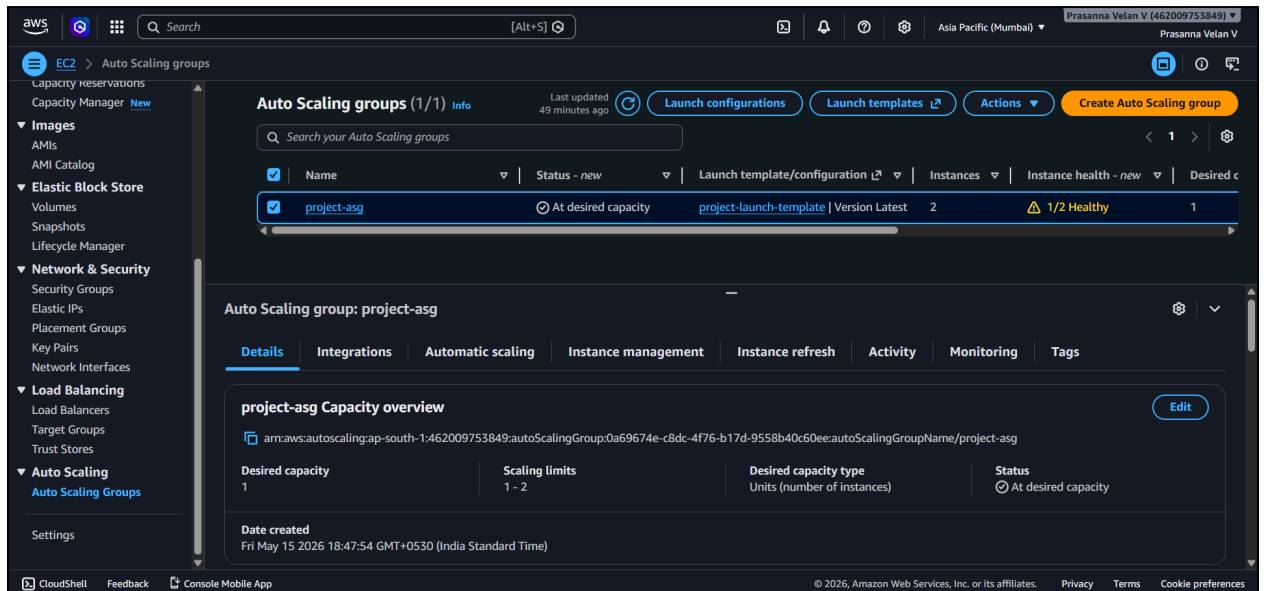
Step 8: Create Launch Template

- Go to EC2 → Launch Templates.
- Configure: AMI Instance Type Security Groups
- User Data Script
 - Automate Apache/PHP application setup using User Data.



Step 9: Create Auto Scaling Group

- Go to Auto Scaling Groups → Create ASG.
- Attach Launch Template. Select: Public Subnet 1 ,Public Subnet 2
- Attach Target Group.
- Configure: Desired Capacity = 1 ,Minimum Capacity = 1 ,Maximum Capacity = 2



Step 10: Verify Auto Scaling and Load Balancing

- Access application using ALB DNS endpoint.
- Verify: Load Balancer, health checks, EC2 instance health, Database connectivity ,S3 image loading
- Terminate one EC2 instance to test automatic recovery.

Scalable Web Application with Load Balancing

The screenshot shows the AWS Management Console for the 'Target groups' page. The left sidebar contains navigation links for EC2, Capacity Reservations, Capacity Manager, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area displays 'Target groups (1/1)' with a table listing the 'project-target-group'. Below the table, the 'Registered targets (3)' section shows a list of targets, including 'web-server' with a health status of 'Healthy'.

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
project-target-group	arn:aws:elasticloadbalancing...	80	HTTP	Instance	project-alb	vpc-08689...

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...	Overri...
i-0b696672ab120793f	web-server	80	ap-south-1a...	Healthy	-	No override.	No over

The screenshot shows the AWS Management Console for the 'Instances' page. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Capacity Manager, Images, Elastic Block Store, Network & Security, and Security Groups. The main content area displays 'Instances (1/3)' with a table listing the 'web-server' instance. Below the table, the 'Details' section shows the instance's configuration, including the Instance ID, IPv6 address, Hostname type, Public IPv4 address, Instance state, Private IP DNS name, and Private IP addresses.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 Df
web-server	i-0b696672ab120793f	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	-
	i-0d90b01f35cac291	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	-
	i-0b38866c1592cb18f	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	-

i-0b696672ab120793f (web-server)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary

Instance ID: i-0b696672ab120793f

IPv6 address: -

Hostname type: IP name: ip-10-0-1-252.ap-south-1.compute.internal

Public IPv4 address: 13.206.197.76 | open address

Instance state: Running

Private IP DNS name (IPv4 only): ip-10-0-1-252.ap-south-1.compute.internal

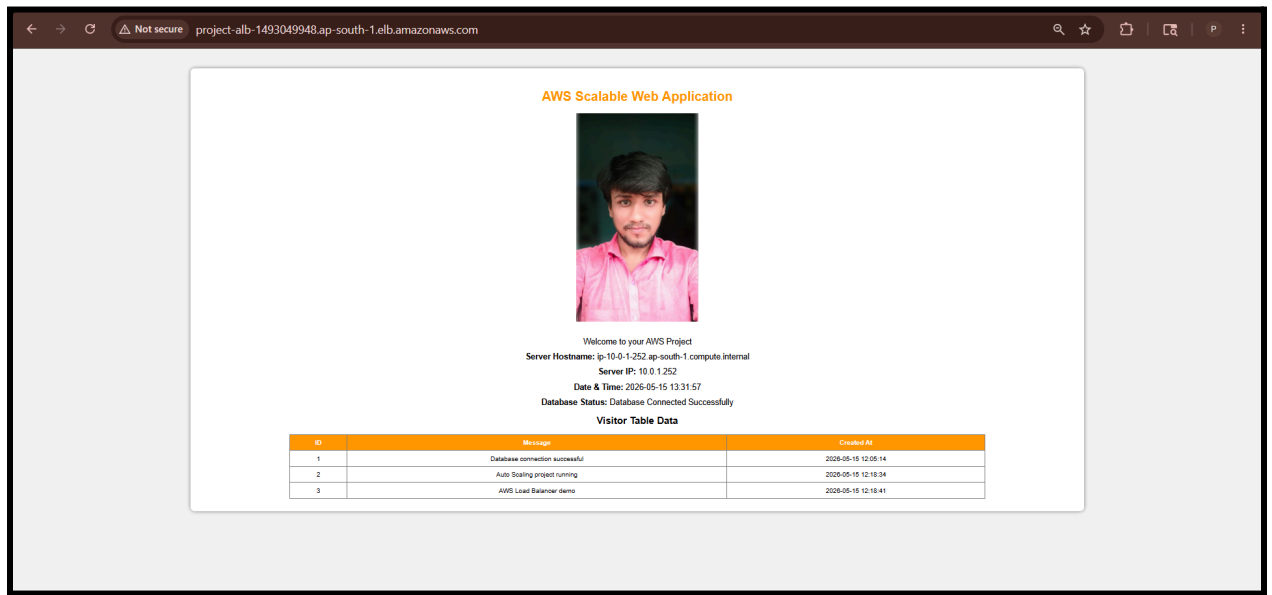
Private IP addresses: 10.0.1.252

Public DNS: -

```
MySQL [(none)]> USE projectdb;
Database changed
MySQL [projectdb]> CREATE TABLE visitors (
  ->   id INT AUTO INCREMENT PRIMARY KEY,
  ->   message VARCHAR(255),
  ->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.059 sec)

MySQL [projectdb]> INSERT INTO visitors (message)
  -> VALUES ('Database connection successful');
Query OK, 1 row affected (0.028 sec)

MySQL [projectdb]> SELECT * FROM visitors;
+----+-----+-----+
| id | message                               | created_at |
+----+-----+-----+
| 1  | Database connection successful         | 2026-05-15 12:05:14 |
+----+-----+-----+
1 row in set (0.002 sec)
```



Troubleshooting:

Issue 1: Target Group Health Status Unhealthy

The Application Load Balancer marked EC2 instances as unhealthy because Apache Web Server was not installed properly through User Data scripts.

Solution:

- Verify User Data script configuration.
- Check Apache installation.
- Verify HTTP service status: `sudo systemctl status httpd`
- Verify Security Group rules for ALB and EC2.

Issue 2: User Data Script Failed

The Launch Template used Amazon Linux 2 while the script used the `dnf` package manager.

Solution:

- Replace `dnf` commands with `yum` commands in User Data script.
- Create new Launch Template version.
- Update Auto Scaling Group with latest template version.

Issue 3: Database Connection Failed

The PHP application could not connect to Amazon RDS due to incorrect Security Group configuration.

Solution:

- Allow MySQL port 3306 from web server Security Group.
- Verify RDS endpoint, username, and password.

Issue 4: S3 Object Access Denied

The uploaded image could not be displayed because the S3 object was not public.

Solution:

- Configure object public access permissions.
- Verify correct S3 object URL.

Conclusion:

The Scalable Web Application with Load Balancing project successfully demonstrates a cloud-native scalable architecture using AWS services. The project integrates EC2, Application Load Balancer, Auto Scaling, Amazon RDS, and Amazon S3 to provide a highly available, scalable, and automated web application infrastructure.

The use of Launch Templates and User Data scripts enables automated server provisioning, while Auto Scaling ensures fault tolerance and automatic recovery of failed instances. The Application Load Balancer efficiently distributes traffic among healthy instances, improving reliability and scalability.

Amazon RDS provides managed relational database services, while Amazon S3 delivers scalable static file hosting. The architecture is Free Tier optimized and uses the ALB DNS endpoint instead of Route 53 custom domains to minimize additional costs.

This project demonstrates real-world AWS cloud engineering concepts including scalability, high availability, automation, security, load balancing, and infrastructure management.